

AI-DLC Workflow — Worked Example

A phase-by-phase walkthrough using the TaskFlow sample project

August 2026

Contents

Introduction	1
Greenfield vs. Brownfield	1
The Folder Structure	2
The Phases	2
Inception	2
Construction	3
Operations	3
Traceability: the thread through every phase	3
How to use this example	4

Introduction

This guide explains the AI-DLC (AI-Driven Development Life Cycle) workflow using a single, coherent sample project called **TaskFlow** — a small team task-management application. It is a companion to the `example/` folder, which contains one illustrative Markdown artifact per phase, laid out in the canonical `aidlc-docs/` directory structure.

AI-DLC is an adaptive software-development workflow from AWS. It guides a project through three lifecycle stages:

- **Inception** — determines *what* to build and *why*.
- **Construction** — determines *how* to build it.
- **Operations** — deploys, runs, and validates the system in production.

The workflow keeps you in control: each phase produces a reviewable artifact, and later phases build on earlier ones.

Greenfield vs. Brownfield

Before presenting the phase list, AI-DLC detects the project type:

- **Greenfield** — no existing source or build files. The workflow has **12 phases**, starting at *Requirements analysis and validation*.
- **Brownfield** — existing code is found. The workflow adds **Reverse Engineering** at the top, for **13 phases** total.

TaskFlow is treated as **brownfield** in this example (it had an undocumented v1 codebase), so the guide covers all 13 phases. A greenfield project simply skips Phase 1 and rennumbers the rest 1–12.

The Folder Structure

Every phase writes its artifact into a predictable location under `aidlc-docs/`:

```
example/
|-- README.md
|-- AI-DLC-Example-Guide.pdf
`-- aidlc-docs/
    |-- aidlc-state.md
    |-- inception/
    |   |-- reverse-engineering/01-reverse-engineering.md
    |   |-- requirements/02-requirements.md
    |   |-- user-stories/03-user-stories.md
    |   |-- design/04-application-design.md
    |-- construction/
    |   |-- units-of-work/05-units-of-work.md
    |   |-- risk-assessment/06-risk-assessment.md
    |   |-- component-design/07-component-design.md
    |   |-- code-generation/08-code-generation.md
    |   |-- build-config/09-build-config.md
    |   |-- qa/10-quality-assurance.md
    |-- operations/
    |   |-- deployment/11-deployment.md
    |   |-- monitoring/12-monitoring.md
    |-- production-readiness/13-production-readiness.md
```

The `aidlc-state.md` file tracks the project summary, detected type, and the status of every phase — it is the workflow’s source of truth for “where are we?”.

The Phases

Inception

Phase 1 — Reverse Engineering (brownfield only). Analyzes the existing codebase to reconstruct requirements, architecture, and a domain model. For TaskFlow, this surfaced two critical gaps in v1: in-memory sessions (blocking horizontal scaling) and no pagination on the task list.

Phase 2 — Requirements analysis and validation. Captures goals, functional requirements (FR), and non-functional requirements (NFR). Each requirement is testable and traceable. TaskFlow’s must-haves include login, projects, tasks, board moves, pagination, and stateless scaling.

Phase 3 — User story creation. Converts requirements into user stories with acceptance

criteria, using the “*As a role, I want capability, so that benefit*” form. Stories map back to the FR/NFR IDs from Phase 2.

Phase 4 — Application Design. Defines the target architecture and records key decisions (ADR-style). TaskFlow moves sessions to Redis, adds a background worker for reminders, keeps PostgreSQL, and serves the SPA from a CDN.

Construction

Phase 5 — Creating units of work for parallel development. Breaks the design into independent units (Auth, Projects, Tasks, Board UI, Reminders) with explicit interface contracts so multiple teams can build in parallel.

Phase 6 — Risk assessment and complexity evaluation. Builds a risk register with likelihood, impact, and mitigations, and rates each unit’s complexity. The session migration and reminder worker are the high-risk, high-complexity items.

Phase 7 — Detailed component design. Specifies each component’s endpoints, data shapes, rules, and error handling in enough detail to implement — for example the paginated Tasks API and the idempotent reminder worker.

Phase 8 — Code generation and implementation. Implements the components, mapping design to source files. The artifact records the file map and representative snippets, keeping traceability back to requirements.

Phase 9 — Build configuration and testing strategies. Sets up the build (npm workspaces, Docker), the test pyramid (unit, integration, E2E), and a CI pipeline with quality gates.

Phase 10 — Quality assurance and validation. Runs the tests and reviews, validates each requirement, checks security, and records the verdict. TaskFlow’s MVP acceptance criteria all pass.

Operations

Phase 11 — Deployment automation and infrastructure. Provisions infrastructure as code (ECS Fargate, RDS, ElastiCache, CloudFront) and automates a blue/green deploy pipeline with migrations and rollback.

Phase 12 — Monitoring and observability setup. Establishes logs, metrics, and traces; defines SLIs, dashboards, and alerts; and correlates them via a shared request ID.

Phase 13 — Production readiness validation. Runs the go-live checklist, publishes SLOs, writes the on-call runbook, plans a staged launch, and collects sign-off.

Traceability: the thread through every phase

The power of AI-DLC is that artifacts connect. Following a single requirement through TaskFlow:

- **FR-6 (pagination)** is defined in *Requirements* (Phase 2).
- It appears as acceptance criteria in *User Stories* (Phase 3).
- It shapes the keyset-cursor design in *Component Design* (Phase 7).
- It is implemented in the task-list snippet in *Code Generation* (Phase 8).
- It is verified with a 500-task integration test in *QA* (Phase 10).

This chain means every line of production behavior can be traced back to a reason, and every requirement can be traced forward to evidence that it works.

How to use this example

1. Read this guide for the narrative.
2. Open `aidlc-docs/aidlc-state.md` to see the phase tracker.
3. Walk the artifacts in numeric order (01 $\rightarrow$ 13), noticing how each phase consumes the outputs of the ones before it.
4. When you run AI-DLC on your own project, expect the same structure — the agent detects greenfield vs. brownfield, asks which phase to start from, gathers any existing context, and then produces artifacts in these same locations.